Assignment No.1

Title: Implementation of Tokenization, Stemming, and Lemmatization on a Small Text Corpus

Aim: To implement Tokenization, Stemming, and Lemmatization techniques on a small corpus of text using Python.

Objectives:

1. To understand the concept of text preprocessing in Natural Language Processing (NLP).
2. To implement tokenization on a given text corpus.
3. To apply stemming techniques to reduce words to their root form.
4. To apply lemmatization to obtain meaningful base forms of words.
5. To compare stemming and lemmatization results.

Expected Outcomes:

1. Successful tokenization of a given text corpus into words.
2. Correct implementation of stemming using appropriate algorithms.
3. Correct implementation of lemmatization using lexical knowledge.
4. Clear understanding of differences between stemming and lemmatization.
5. Improved knowledge of basic NLP preprocessing techniques.

Theory

1. Introduction to Natural Language Processing (NLP)

Natural Language Processing (NLP) is a field of Artificial Intelligence that enables computers to understand, interpret, and process human language.

NLP is widely used in:

- Chatbots
- Sentiment analysis
- Machine translation
- Text classification
- Search engines

Before performing advanced NLP tasks, text must be preprocessed.

2. Text Preprocessing

Text preprocessing is the process of cleaning and preparing raw text data for analysis.

It includes:

- Tokenization
- Stemming
- Lemmatization
- Removing stopwords
- Lowercasing

These steps help improve model performance and accuracy.

3. Tokenization

Tokenization is the process of breaking text into smaller units called tokens.

Tokens can be:

- Words
- Sentences
- Characters

Example:

Input Text:
"I am learning Natural Language Processing."

Word Tokens:
I, am, learning, Natural, Language, Processing

Tokenization is the first step in most NLP tasks.

4. Stemming

Stemming reduces a word to its root or base form by removing suffixes.

It may not always produce meaningful words.

Example:

- playing → play
- studies → studi
- running → run

Stemming is faster but less accurate.

Common stemming algorithms:

- Porter Stemmer
- Snowball Stemmer

5. Lemmatization

Lemmatization reduces a word to its meaningful base form (lemma) using vocabulary and morphological analysis.

It returns valid dictionary words.

Example:

- running → run
- better → good
- studies → study

Lemmatization is slower than stemming but more accurate.

6. Difference Between Stemming and Lemmatization

Stemming and lemmatization are both text normalization techniques used in Natural Language Processing, but they differ in accuracy and approach. Stemming simply removes suffixes from words to produce a root form, and the output may not always be a meaningful word; for example, *"studies"* becomes *"studi."* It is faster and computationally efficient but less accurate. In contrast, lemmatization uses vocabulary knowledge and linguistic rules to convert words into their correct dictionary base form, ensuring meaningful results; for example, *"studies"* becomes *"study."* Although lemmatization is slower than stemming, it is more accurate and produces linguistically correct outputs.

Pseudocode:

```
START

INPUT:
Small text corpus

STEP 1: Import Required Libraries
Import NLTK library
Download necessary datasets (punkt, wordnet)

STEP 2: Preprocess Text
Convert text to lowercase

STEP 3: Perform Tokenization
Split text into word tokens

STEP 4: Perform Stemming
Initialize Porter Stemmer
Apply stemming to each token
Store stemmed words

STEP 5: Perform Lemmatization
Initialize WordNet Lemmatizer
Apply lemmatization to each token
Store lemmatized words

OUTPUT:
Original Tokens
Stemmed Words
Lemmatized Words

END
```

Algorithm:

1. Start the program.
2. Input the text corpus.
3. Convert the text into lowercase.
4. Tokenize the text into words.
5. Apply stemming to each word.
6. Apply lemmatization to each word.
7. Display the results.
8. Stop the program.

Results:

1. **Word Tokenizer**

```
words=nltk.word_tokenize(paragraph)

print(words)
```

```
['Hello', 'world', 'good', 'morning', 'have', 'a', 'great', 'day.A', 'paragraph', 'is', 'a', 'collection', 'of', 'sentences', 'focused',
```

2. **Sentence Tokenizer**

['Hello world good morning have a great day.A paragraph is a collection of sentences focused on a single main idea.', 'Typically, effective paragraphs include aparagragh to introduce the central point, several supporting sentences for details, and a concluding sentence to summarize or transition']

3. **Poter Stemmer**

```
Hello---->hello
world---->world
good---->good
morning---->morn
have---->have
a---->a
great---->great
day.A---->day.a
paragraph---->paragraph
is---->is
a---->a
collection---->collect
of---->of
sentences---->sentenc
focused---->focus
on---->on
a---->a
single---->singl
main---->main
idea---->idea
.---->.
```

4. **Lemmatizer**

```
Hello--->Hello
world--->world
good--->good
morning--->morning
have--->have
a--->a
great--->great
day.A--->day.A
paragraph--->paragraph
is--->is
a--->a
collection--->collection
of--->of
sentences--->sentence
focused--->focused
on--->on
a--->a
single--->single
main--->main
idea--->idea
.--->.
Typically--->Typically
,--->,
effective--->effective
paragraphs--->paragraph
include--->include
aparagragh--->aparagragh
to--->to
```

Conclusion

Tokenization, stemming, and lemmatization are essential preprocessing steps in Natural Language Processing. Tokenization breaks text into meaningful units, stemming reduces words to root forms, and lemmatization converts words into their meaningful base forms. Among the two normalization techniques, lemmatization provides more accurate results, while stemming is computationally faster. These techniques form the foundation for advanced NLP applications such as text classification, sentiment analysis, and language modelling.